End-to-End Data Management Pipeline for a Recommendation System

Data Management for Machine Learning - Assignment 2

Akshay Agrawal | Roll No: 2025AG05201

GitHub Repository
https://github.com/AKbhai18/recommendation-pipeline

1. Overview and Design Decisions

This project implements a modular, reproducible data-management pipeline for a MovieLens-style movie recommendation system. It covers the full lifecycle: data ingestion, validation, preparation, transformation, feature management, model training, evaluation, and orchestration. The emphasis is a clean, modular, reproducible pipeline rather than only maximising accuracy.

Key design decisions

- Config-driven and seeded: all parameters live in a single configs/config.yaml and one global seed makes every run deterministic and reproducible.
- Validation precedes preparation: ingestion is intentionally lenient so the validation stage observes and reports the data exactly as it arrived, quarantining bad rows with a reason.
- Time-based train/test split: the most recent portion of each user's history is held out, mimicking the real task of predicting future interactions and avoiding data leakage.
- A reusable feature store: user, item and interaction features are computed from the training split only and persisted, so the identical definitions serve both training and prediction.
- Two orchestration options: a dependency-free Python DAG runner for easy reproduction, plus an equivalent Apache Airflow DAG.

Tools used

Python, pandas and NumPy for data handling; Pandera for schema and data-quality validation; scikit-learn (TruncatedSVD, cosine similarity) and SciPy for modelling; MLflow for experiment tracking; Parquet/CSV for storage; and Apache Airflow (plus a structured Python workflow) for orchestration. Matplotlib is used for the diagram and figures.

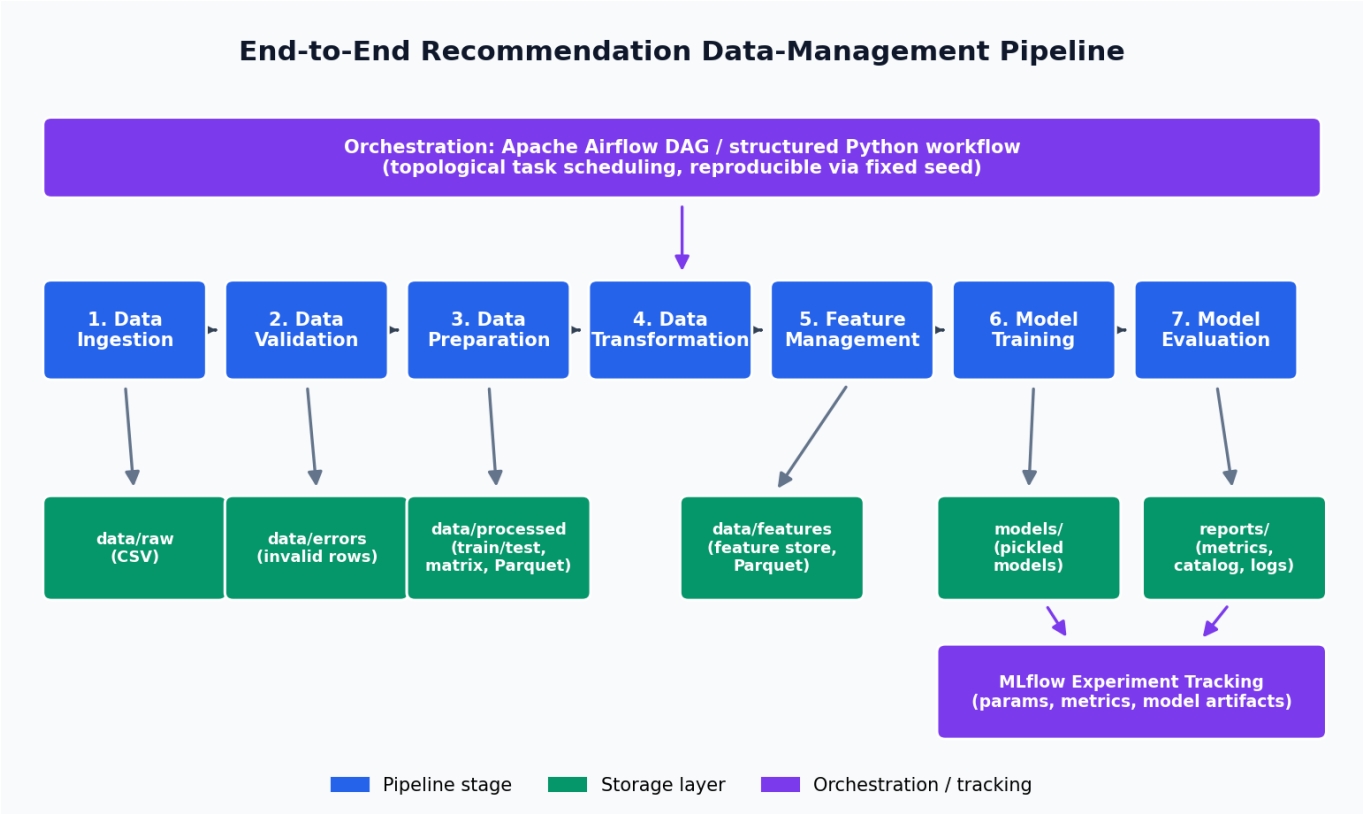


Figure 1. Pipeline architecture - eight stages, the storage layers they read/write, MLflow tracking, and the orchestration layer.

2. Data, Ingestion and Validation

The pipeline generates a documented synthetic dataset modelled on MovieLens (users, items, and explicit 1-5 interaction ratings). Interactions are drawn from a latent-factor taste model so there is genuine collaborative structure for the personalised models to recover. A small, configurable fraction of rows is deliberately corrupted (missing/out-of-range ratings, invalid timestamps, dangling foreign keys, duplicates) to exercise the validation stage. Real datasets can be used by placing matching CSVs in data/raw/.

The validation stage combines Pandera schema checks on the dimension tables with custom row-level checks on the interactions. Every invalid row is written to data/errors/invalid_interactions.csv with a per-row reason, and a summary report is produced. In the run shown below, 8,716 of 61,800 interaction rows (14.1%) were quarantined across all the injected problem types.

Stage 1 - Data Ingestion

```
$ python -m src.ingestion.ingest
2026-11-10 13:09:49 | INFO    | ingestion.ingest   | Ingested users.csv        rows=1000 cols=5
2026-11-10 13:09:49 | INFO    | ingestion.ingest   | Ingested items.csv        rows=500 cols=4
2026-11-10 13:09:49 | INFO    | ingestion.ingest   | Ingested interactions.csv rows=61800 cols=5
2026-11-10 13:09:49 | INFO    | ingestion.ingest   | Ingestion complete: {'users': 1000, 'items': 500, 'interactions':
61800}

== users ==
  user_id age gender  occupation signup_date
0   U1000  38      U     retired  2023-04-27
1   U1001  42      M     retired  2022-04-26
2   U1002  62      M  technician  2023-07-29
3   U1003  22      F       other  2023-02-02
4   U1004  41      F      doctor  2021-04-14

== items ==
  item_id              title     category release_year
0   I2000  Electric Voyage 2        SciFi         1999
1   I2001   Golden Promise 3      Romance         2002
2   I2002   Silent Horizon 2  Documentary         2022
3   I2003  Midnight Empire 2       Horror         2018
4   I2004  Crimson Kingdom 1       Comedy         2004

== interactions ==
  user_id item_id rating            timestamp event_type
0   U1931   I2486    3.0  2023-01-28 01:51:57     rating
1   U1699   I2046    4.5  2021-08-25 03:26:50     rating
2   U1259   I2053    5.0  2021-12-19 04:38:03     rating
3   U1081   I2140    5.0  2021-02-22 13:23:00      click
4   U1676   I2088    5.0  2022-08-21 00:37:50       view
```

Figure 2. Stage 1 - data ingestion of users, items and interactions.

Stage 2 - Data Validation

```
$ python -m src.validation.validate
2026-11-10 13:09:51 | INFO    | ingestion.ingest   | Ingested users.csv        rows=1000 cols=5
2026-11-10 13:09:51 | INFO    | ingestion.ingest   | Ingested items.csv        rows=500 cols=4
2026-11-10 13:09:51 | INFO    | ingestion.ingest   | Ingested interactions.csv rows=61800 cols=5
2026-11-10 13:09:51 | INFO    | ingestion.ingest   | Ingestion complete: {'users': 1000, 'items': 500, 'interactions':
61800}
2026-11-10 13:09:51 | INFO    | validation.validate | Validating dimension tables ...
2026-11-10 13:09:51 | INFO    | validation.validate | [users] schema validation PASSED (rows=1000)
2026-11-10 13:09:51 | INFO    | validation.validate | [items] schema validation PASSED (rows=500)
2026-11-10 13:09:51 | INFO    | validation.validate | Validating interactions (row-level) ...
2026-11-10 13:09:51 | INFO    | validation.validate | Validation done: clean=53084 quarantined=8716 (14.10% bad)
{
  "dimensions": [
    {
      "table": "users",
      "rows": 1000,
      "schema_ok": true,
      "errors": 0
    },
    {
      "table": "items",
      "rows": 500,
      "schema_ok": true,
      "errors": 0
    }
  ],
  "interactions": {
    "total_rows": 61800,
    "clean_rows": 53084,
    "error_rows": 8716,
    "missing_rating": 1809,
    "rating_out_of_range": 1854,
    "invalid_timestamp": 1848,
    "unknown_user_id": 0,
    "unknown_item_id": 1855,
    "duplicate_interaction": 1595
  }
}
```

Figure 3. Stage 2 - data validation: schema checks and row-level quarantining of invalid interactions.

3. Preparation, Transformation and Features

Preparation standardises formats, removes cold users with too few interactions, performs the per-user time-based split, and builds the derived datasets: a user activity summary, item popularity, and the user-item rating matrix (built from the training split only). The feature layer then computes reusable user, item and interaction features and persists them as a Parquet feature store.

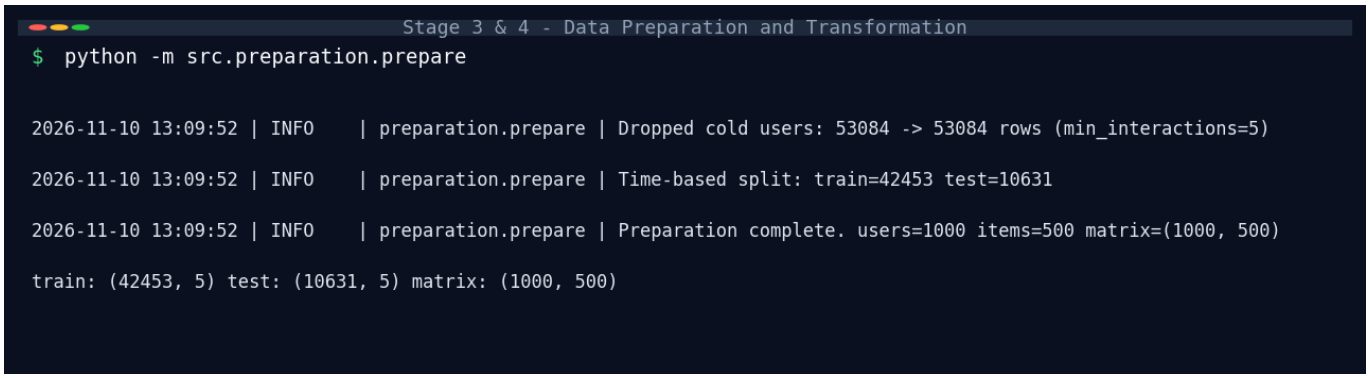


Figure 4. Stages 3 & 4 - preparation and transformation: cleaning, time-based split, and derived datasets.

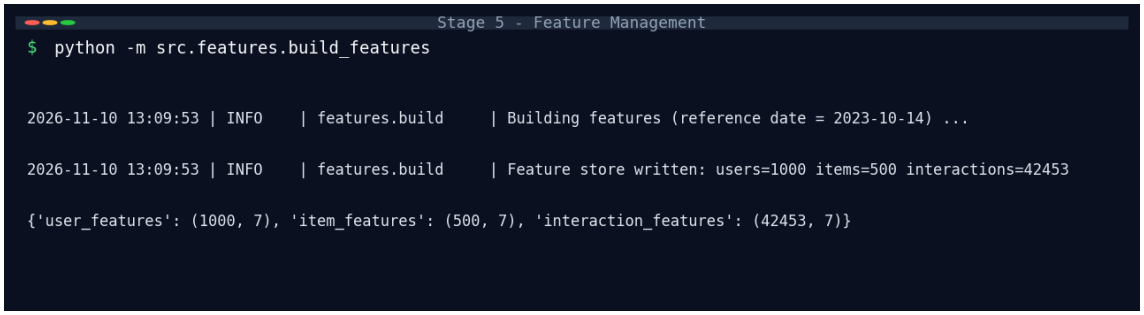


Figure 5. Stage 5 - feature management: the persisted feature store.

Feature catalog

Entity	Feature	Description
user	user_avg_rating	Mean rating the user has given across all training interactions.
user	user_activity_count	Number of interactions the user has in the training data.
user	user_distinct_items	Number of distinct items the user interacted with.
user	user_rating_std	Standard deviation of the user's ratings (rating consistency).
user	user_days_since_last	Days between the user's last interaction and the reference date.
user	user_preferred_category	Item category the user interacts with most often.
item	item_popularity	Number of interactions the item received (interaction count).
item	item_distinct_users	Number of distinct users who interacted with the item.
item	item_avg_rating	Mean rating the item received.
item	item_rating_std	Standard deviation of the item's ratings.
item	item_release_year	Release year of the item (from the item catalog).
item	item_category	Category of the item (from the item catalog).
interaction	rating	Explicit rating given in the interaction (target for RMSE/MAE).
interaction	event_type	Type of interaction: rating / view / click / purchase.
interaction	interaction_month	Calendar month of the interaction (1-12), a seasonality signal.
interaction	interaction_dayofweek	Day of week of the interaction (0=Mon..6=Sun).

4. Model Training and Evaluation

Three models are trained and compared: a non-personalised popularity baseline, item-based collaborative filtering (cosine item-item similarity), and a matrix-factorisation model (truncated SVD on the mean-centred user-item matrix). Every run's parameters, metrics and serialised model are logged to MLflow. Models are evaluated on the held-out time-based test split using Precision@K, Recall@K, NDCG@K, Coverage and Diversity (K=10), plus RMSE and MAE for rating prediction.

Model	P@10	R@10	NDCG	Cov.	Div.	RMSE	MAE
popularity baseline	0.022	0.033	0.030	0.044	0.869	0.707	0.503
item based cf	0.057	0.081	0.075	0.942	0.873	0.726	0.440
matrix factorization svd	0.043	0.061	0.062	0.620	0.871	0.669	0.443

Both personalised models clearly outperform the popularity baseline on ranking quality. Item-based collaborative filtering achieves the best top-K ranking (Precision@10 of 0.057 versus 0.022 for the baseline) and by far the widest catalog coverage (0.94 versus 0.04), because the baseline recommends the same popular items to everyone. Matrix factorisation gives the best explicit-rating prediction (lowest RMSE).

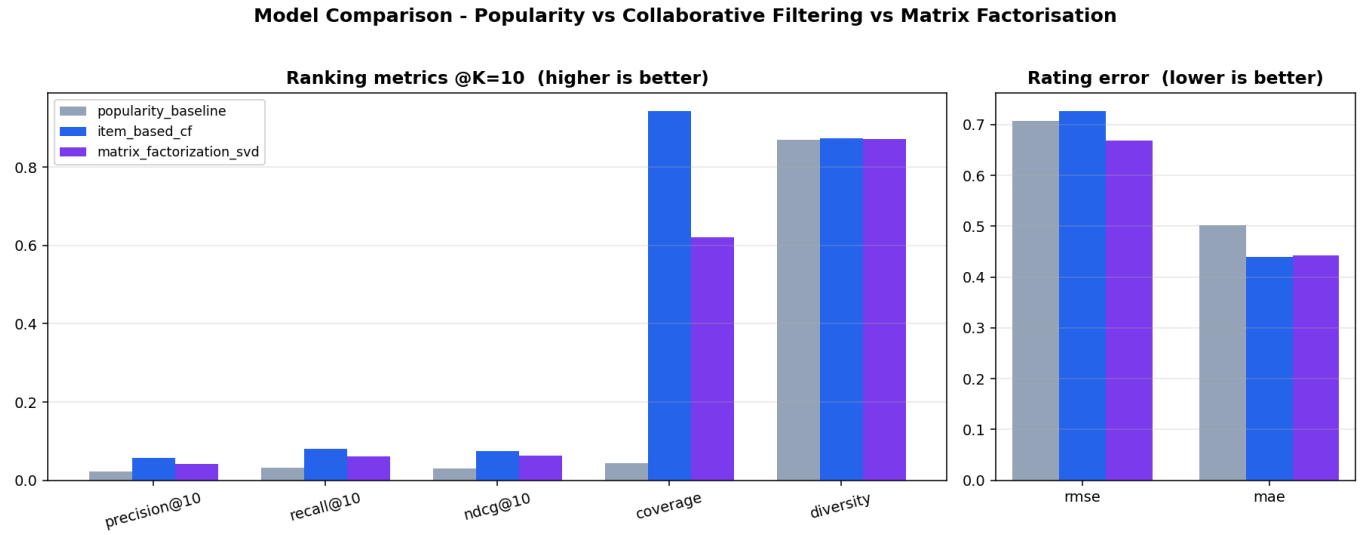


Figure 6. Model comparison across ranking metrics and rating error.

Stage 6 & 7 - Model Training and Evaluation

```
$ python -m src.training.train

2026-11-10 13:09:56 | INFO     | training.train     | Training model: popularity_baseline
2026-11-10 13:09:57 | INFO     | evaluation.metrics | [popularity_baseline] {'precision@10': 0.0224, 'recall@10':
0.033, 'ndcg@10': 0.0298, 'coverage': 0.044, 'diversity': 0.8687, 'eval_users': 998, 'rmse': 0.7074, 'mae': 0.5026}
2026-11-10 13:09:57 | INFO     | training.train     | Training model: item_based_cf
2026-11-10 13:09:59 | INFO     | evaluation.metrics | [item_based_cf] {'precision@10': 0.0574, 'recall@10': 0.0806,
'ndcg@10': 0.0746, 'coverage': 0.942, 'diversity': 0.8734, 'eval_users': 998, 'rmse': 0.7257, 'mae': 0.4398}
2026-11-10 13:10:00 | INFO     | training.train     | Training model: matrix_factorization_svd
2026-11-10 13:10:01 | INFO     | evaluation.metrics | [matrix_factorization_svd] {'precision@10': 0.0425, 'recall@10':
0.0615, 'ndcg@10': 0.0623, 'coverage': 0.62, 'diversity': 0.8705, 'eval_users': 998, 'rmse': 0.6691, 'mae': 0.4426}
2026-11-10 13:10:01 | INFO     | training.train     | Training complete. Results:
                  model  precision@10  recall@10  ndcg@10  coverage  diversity  eval_users   rmse    mae
     popularity_baseline        0.0224     0.0330   0.0298     0.044     0.8687         998 0.7074 0.5026
           item_based_cf        0.0574     0.0806   0.0746     0.942     0.8734         998 0.7257 0.4398
matrix_factorization_svd        0.0425     0.0615   0.0623     0.620     0.8705         998 0.6691 0.4426
                  model  precision@10  recall@10  ndcg@10  coverage  diversity  eval_users   rmse    mae
     popularity_baseline        0.0224     0.0330   0.0298     0.044     0.8687         998 0.7074 0.5026
           item_based_cf        0.0574     0.0806   0.0746     0.942     0.8734         998 0.7257 0.4398
matrix_factorization_svd        0.0425     0.0615   0.0623     0.620     0.8705         998 0.6691 0.4426
```

Figure 7. Stages 6 & 7 - training and evaluation console output with the comparison table.

MLflow Experiment: 'movielens_recsys' - 3 tracked runs (tracking URI: file:reports/mlruns)

run_name	precision@10	recall@10	ndcg@10	coverage	rmse	mae
atrix_factorization_s	0.0425	0.0615	0.0623	0.62	0.6691	0.4426
item_based_cf	0.0574	0.0806	0.0746	0.942	0.7257	0.4398
opularity_baseline	0.0224	0.033	0.0298	0.044	0.7074	0.5026

Figure 8. MLflow experiment tracking - the three logged runs with their parameters and metrics.

5. Orchestration and Reproducibility

The complete workflow is automated as a directed acyclic graph with correct task dependencies: generate_data -> ingest -> validate -> prepare -> build_features -> train_and_evaluate. The structured Python orchestrator performs a topological sort and runs each stage in dependency order with timing, while an equivalent Apache Airflow DAG is provided for scheduled production use. Because every parameter is config-driven and seeded, the end-to-end run is fully reproducible.



Figure 9. Stage 8 - a successful end-to-end orchestrated run, showing each stage completing in dependency order.

6. Limitations and Future Work

- The dataset is synthetic. Although modelled on MovieLens with a realistic latent-taste structure, absolute metric values differ from a real dataset; the pipeline accepts real CSVs unchanged.
- The collaborative-filtering and matrix-factorisation models are classical. Stronger models (e.g.

implicit-feedback ALS, LightFM, or neural recommenders) would likely improve ranking quality.

- Absolute Precision@K values are modest because the catalog is large (500 items) and each user's test set is small, so the relative comparison between models is the key result.
- The feature store is file-based. A production system would use a dedicated store (e.g. Feast) with online serving and point-in-time correctness.
- Validation thresholds are fixed in config; a production setup would add alerting and quarantine-rate monitoring over time.